Distributed Databases

Sharding

- Splitting a large database into smaller pieces across multiple servers. Each 'shard' stores :-
  - A portion of total data

Instead of one big database with 100 million users :-
- we might have :- shard 1 → 1 - 10 M users
  shard 2 → 10 - 20 M users
- each runs independently.

- Sharding allows horizontal scaling, load distribution & helps solve :-
  - storage limits
  - High write load
  - High read load
  - scalability bottleneck.

Working

- A request comes — GET user with id = 1234.
- System must decide which shard contains this user?
- The decision is based on 'shard key'. It determines where data lives.

Shard key

- shard key is :-
  - The column must used to decide how data is distributed.

  common shard keys :-
  user_id, region, customer_id, tenant_id.

- For backend
  Try Index optimization, query optimization, read replicas, caching.
  sharding is usually last scaling step.

Types of Sharding

1) Range Based :- Data split by ranges (1-2M, 2-3M)

Problem :- If most new users fall in last range that shard becomes overloaded. This is called hotspot.

2) Hash Based :- Data distributed using hash function. This distributes data evenly which ensures good load balancing

Problem :- Rebalancing is complex, Harder to do range queries.

3) Directory Based :- A lookup table tells 'which user belongs to which shard'. It is more flexible but adds extra lookup step, More management.

Cross-shard Query Problem

Ex :- SELECT * from users;

- If users are in 10 shards; you must :-
  - Query all shards & combine results.

Increases complexity & latency.

When to Use ?

- Vertical scaling is insufficient
- Data size is massive
- Write throughput is high
- Database becomes bottleneck

So as mentioned earlier (last page), do not shard too early. Keep it as last scaling step.